

Lab: Setting Up the Active Inference Toolkit

Learning Goal: Implement core data structures and utility functions for Active Inference in Python.

Part 1: Environment Setup

Exercise: Set up your Python environment and verify it works.

```
# Run this to verify your setup
import numpy as np
print(f"NumPy version: {np.__version__}")

# Create a simple probability distribution
p = np.array([0.3, 0.5, 0.2])
print(f"Distribution: {p}")
print(f"Sums to: {p.sum()}") # Should be 1.0
```

Write your response here

Part 2: Implement Utility Functions

Learning Goal: Code the mathematical building blocks.

Exercise: Implement these functions and test them:

```

def normalize(x):
    """Normalize a vector to sum to 1."""
    # YOUR CODE HERE
    pass

def log_stable(x, eps=1e-16):
    """Numerically stable log."""
    # YOUR CODE HERE
    pass

def softmax(x):
    """Convert log-probabilities to probabilities."""
    # YOUR CODE HERE
    pass

def entropy(p):
    """Compute entropy  $H(p) = -\sum(p * \log(p))$ ."""
    # YOUR CODE HERE
    pass

def kl_divergence(q, p):
    """Compute KL divergence  $D_{KL}(q || p)$ ."""
    # YOUR CODE HERE
    pass

# Test cases
p = np.array([0.5, 0.3, 0.2])
q = np.array([0.4, 0.4, 0.2])

print(f"Entropy of p: {entropy(p):.4f}") # ~1.03

```

```
print(f"KL(q || p): {kl_divergence(q, p):.4f}") # should be > 0
print(f"KL(p || p): {kl_divergence(p, p):.4f}") # should be ~0
```

Write your response here

Part 3: Build a Generative Model

Learning Goal: Create the A, B, C, D data structures for a specific scenario.

Scenario: A foraging agent.

- States: {food_left, food_right} (2 states)
- Observations: {see_food, see_nothing} (2 observations)
- Actions: {go_left, go_right} (2 actions)
- Preference: Agent prefers seeing food
- Define A: The agent has 80% chance of correctly observing food at its location
- Define B: go_left reliably moves to left state, go_right to right state
- Define C: Preference for see_food over see_nothing
- Define D: Uniform initial belief
- Validate all matrices

```
# YOUR CODE HERE
A = np.array(...) # (2, 2)
B = np.zeros((2, 2, 2))
B[:, :, 0] = ... # go_left
B[:, :, 1] = ... # go_right
C = np.array(...)
D = np.array(...)
```

Write your response here

Part 4: Bayesian Update in Code

Learning Goal: Implement belief updating.

Exercise: Given your foraging model, implement a single Bayesian update:

```
def bayesian_update(A, prior, observation):
    """
    Compute posterior = normalize(likelihood * prior)

    Parameters:
        A: observation model (num_obs x num_states)
        prior: prior beliefs over states (num_states,)
        observation: integer index of observation

    Returns:
        posterior: updated beliefs over states
    """
    # YOUR CODE HERE
    pass

# Test: Agent starts with uniform prior, observes see_food
prior = D.copy()
posterior = bayesian_update(A, prior, observation=0) # 0 = see_food
print(f"Prior: {prior}")
print(f"Posterior after seeing food: {posterior}")
```

Write your response here

Part 5: Reflection

In 150 words, reflect: What did you learn from implementing the math in code that you didn't understand from the equations alone? Were there any "aha" moments?

Write your response here

Lab Summary

Part	Skill Practiced	Key Concept
1	Environment setup	Python + NumPy
2	Function implementation	normalize, softmax, entropy, KL
3	Model construction	A, B, C, D matrices
4	Inference coding	Bayesian belief update
5	Metacognitive reflection	Learning by implementing